

MINI PROJECT REPORT ON

VERILOG BASED PARTICLE SWARM OPTIMIZATION WITH
DYNAMIC POPULATION SIZE HANDLING FOR AUTO-
TUNING OF RC FILTER



Submitted for the mini project of Digital Circuits Design Laboratory as partial fulfillment
of B.Tech Course

By:

1. Jacob Sadeesh Palet – 24ECB0A24
2. Jalluri Ajitesh – 24ECB0A25
3. Jogu Siddhartha Bixam – 24ECB0A26

Under the guidance of

Prof. Dr. P Prithvi
Prof. Dr. Sourabh Solanki

DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY, WARANGAL
TELANGANA – 506004

October - 2025

**NATIONAL INSTITUTE OF TECHNOLOGY
WARANGAL, INDIA
DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**



CERTIFICATE

This is to certify that the Project work entitled "**Verilog-Based Particle Swarm Optimization with Dynamic Population Size Handling for Auto-Tuning of RC Filter**" is a Bonafide record of work carried out by Jacob Sadeesh Palet-24ECBOA24, Jalluri Ajitesh-24ECB0A25, Jogu Siddhartha Bixam-24ECB0A26 submitted to the faculty of Department of Electronics and Communication Engineering, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering for the course EC1211-Digital Circuits Design Laboratory at National Institute of Technology, Warangal during the academic year 2025-2026.

Prof. P.Prithvi
Associate Professor
NIT WARANGAL
TELANGANA

Prof. Sourabh Solanki
Assistant Professor
NIT WARANGAL
TELANGANA

INDEX

Abstract

Acknowledgement

Table of Contents

1.1 List of Figures

1.2 List of Abbreviations

1.3 List of Tables

1. Introduction

2. Literature Survey

3. Content

3.1 Problem statement

3.2 Motivation

3.3 Novelty

3.4 Implementation flow

3.5 Objectives

4. Theory

5. Discussion and Conclusion

5.1. Result

5.2 Conclusion

5.3. Future work

6. Appendix: Code

7. References

8. Appendix

9. Plagiarism report

ABSTRACT

This work presents a Verilog-based hardware implementation of a Particle Swarm Optimization (PSO) engine designed to automatically tune the cutoff frequency of an RC low-pass filter to a desired target. The system encodes double-precision real values within 64-bit registers, enabling portable arithmetic operations entirely in Verilog. A lightweight uniform pseudo-random number generator (PRNG) provides randomized particle initialization and stochastic coefficients during each iteration.

The PSO algorithm incorporates linearly decaying inertia, cognitive and social acceleration terms, velocity clamping proportional to parameter ranges, and reflecting boundary conditions with damping to maintain feasibility. An adaptive edge penalty discourages convergence at parameter limits when equally valid interior solutions exist. To enhance efficiency, a dynamic particle shutdown heuristic suppresses particles that consistently degrade in performance. Fitness is evaluated as the percentage error between the achieved and target cutoff frequency.

A greedy repair mechanism further accelerates convergence by projecting each PSO update onto the feasible design manifold—computing either C^* for fixed R or R^* for fixed C and selecting the candidate with minimum error after boundary clamping. The implementation executes as a behavioral simulation in Xilinx Vivado, reporting the globally best feasible (R, C) pair after each iteration. Experimental results across wide parameter ranges demonstrate convergence within 5% error for attainable targets, while infeasible cases yield the best possible solution along with explicit residual error. The proposed architecture offers a compact, reproducible, and simulator-portable optimization core suitable for mixed-signal auto-tuning applications.

Index Terms— Particle Swarm Optimization, Verilog HDL, RC Filter, Auto-Tuning, Random Number Generation, Behavioural Simulation, Cutoff Frequency.

ACKNOWLEDGMENT

With great respect and gratitude, I would like to convey my deepest appreciation to everyone who has played a role in making this project a success.

I would like to express my deepest gratitude to Prof. P. Prithvi, Associate Professor in the Department of Electronics and Communication Engineering at the National Institute of Technology, Warangal, for her excellent guidance, continuous support, and really good ideas throughout this project. Her skill and positive remarks have been a big help in forming the technical path and the quality of the project.

I am also very, very grateful to Prof. Sourabh Solanki, Assistant Professor in the Department of Electronics and Communication Engineering at the National Institute of Technology, Warangal, for always being there for me, inspiring me, and giving me valuable inputs at all stages of development. His guidance and technical advice have been a major factor in my growing understanding and doing the project goals.

At last, but not least, I would like to recognize the Electronics and Communication Engineering, NIT Warangal, for granting the necessary that this work was made possible. I also thank my fellow and co-workers for their collaboration, talks and support which were very useful in purifying this project.

TABLE OF CONTENTS

1. List of tables:

Table.1 Parameter Specification for PSO Auto-Tuner	Page 17
Table 2. I/O Signal Description	Page 17
Table 3. Sample Outputs	Page 18

2. List of figures:

Fig. 3.4.1 Implementation Flow Chart	Page 12
Fig. 4.1 Concept of PSO	Page 14
Fig. 4.2 Flow of Particles	Page 15
Fig. 4.3 Functional Diagram	Page 16

3. List of abbreviations:

- PSO - Particle Swarm Optimization
- RNG - Random Number Generation
- DPSH – Dynamic Population Size Handling
- PSNG – Pseudo Random Number Generation
- HDL – Hardware Description Language

CHAPTER-1

INTRODUCTION

Optimization has become a central aspect of modern engineering design, where systems are required to achieve the best possible performance under strict constraints, including cost, time, and resource limitations. In electronics, achieving precise analog characteristics such as filter cutoff frequency, gain, or bandwidth often depends on the careful selection of component values. Manual tuning of these parameters is not only time-consuming but also prone to human error, and the results can vary due to component tolerances, environmental conditions, and manufacturing variations. These challenges create a strong need for automated and intelligent optimization methods that can provide reliable and reproducible design solutions.[1]

Particle Swarm Optimization (PSO) is one such population-based metaheuristic algorithm that has proven effective for complex design problems. Inspired by the collective motion and social behavior of birds and fish, PSO consists of multiple candidate solutions called particles, each exploring the design space simultaneously. Each particle updates its position based on its personal experience and the knowledge of the swarm's global best solution. This social cooperation enables the swarm to rapidly converge toward optimal or near-optimal solutions while avoiding extensive computational overhead typically associated with other evolutionary or gradient-based algorithms. [3]

This project focuses on implementing PSO directly in Verilog Hardware Description Language (HDL) to autonomously tune the cutoff frequency of an RC low-pass filter. The system iteratively searches for the optimal combination of resistance (R) and capacitance (C) values that closely achieve a target cutoff frequency defined by the user. The design supports double-precision calculations using 64-bit encoding, employs a lightweight pseudo-random number generator for particle initialization and stochastic updates, and integrates additional enhancements such as velocity clamping, inertia damping, and dynamic particle shutdown to improve convergence speed and efficiency.

The implementation is verified using behavioral simulation in Xilinx Vivado, with the system continuously reporting the best feasible (R,C) combination at each iteration.[2]

Experimental results demonstrate consistent and reliable convergence with minimal error across a wide range of parameter settings. This work illustrates the potential of hardware-level optimization to create self-tuning, adaptive analog-digital systems, enabling faster, more accurate, and energy-efficient designs in embedded and mixed-signal applications.

CHAPTER-2

LITERACY REVIEW

[1]. M. Ravindran, Mei-Ping Song, and Guo-Chang Gu, “Research on Particle Swarm Optimization: A Review,” IEEE International Conference on Machine Learning and Cybernetics, 2004 [1]:

This foundational paper provides a comprehensive study of PSO theory, parameter tuning, and convergence characteristics. It highlights how population-based intelligence can outperform traditional gradient methods in nonlinear problems. The review also discusses variations of PSO for hardware adaptability — an idea central to this project’s hardware-based RC tuning implementation.

[2]. Yuhui Shi and Russell Eberhart, “Parameter Selection in Particle Swarm Optimization,” Evolutionary Programming VII, Springer, 1998 [2]:

This work focuses on improving PSO stability through inertia weight and constriction factor strategies. The authors demonstrate how dynamic parameter adjustment accelerates convergence without trapping in local minima. These techniques directly influenced this project’s Verilog-based implementation, particularly in maintaining balance between exploration and exploitation across limited precision digital representations.

[3]. J. Kennedy and R. C. Eberhart, “Particle Swarm Optimization,” Proceedings of IEEE International Conference on Neural Networks, 1995 [3]:

The original PSO formulation introduced here remains the theoretical backbone of all swarm optimization systems. Its adaptability to engineering problems has inspired numerous applications in analog and digital domains. This project draws from this model to translate algorithmic PSO behaviour into hardware constructs, effectively enabling analog filter auto-tuning on FPGA or ASIC platforms.

CHAPTER-3

CONTENT

3.1 PROBLEM STATEMENT

Particle Swarm Optimization (PSO) has become a popular technique for tuning parameters in electronic circuits because of its simplicity and strong optimization capabilities. However, most hardware implementations use a fixed swarm size, which often results in unnecessary computations and inefficient use of resources. This static approach can slow down convergence and limit adaptability, especially in dynamic optimization environments. To overcome these limitations, we propose a Verilog-based PSO with Dynamic Population Size Handling (DPSH). This design dynamically adjusts the number of active particles during runtime, allowing the system to adapt its computational effort based on the optimization requirements. By doing so, the approach aims to enhance convergence speed, reduce hardware overhead, and achieve more efficient auto-tuning of electronic circuits.

3.2 MOTIVATION

As modern electronic systems continue to evolve in complexity, there is an increasing demand for circuits capable of autonomously adapting and optimizing their performance without human supervision. In areas such as adaptive filters, communication interfaces, and signal processing modules, even minimal deviations in parameters like the cutoff frequency can noticeably affect system performance and efficiency. However, achieving accurate and real-time tuning remains a significant challenge—traditional manual or fixed-parameter approaches tend to be slow, rigid, and inadequate for the demands of contemporary high-speed systems.

Particle Swarm Optimization (PSO) has emerged as a promising technique for such parameter optimization tasks, valued for its simplicity, scalability, and robustness. Nonetheless, existing hardware implementations typically rely on fixed swarm sizes and static structures, leading to unnecessary computations, greater power usage, and reduced adaptability in dynamic environments. These limitations make conventional PSO architectures less suitable for real-time, energy-aware systems.

To address these issues, this project introduces a Verilog-based PSO engine with dynamic population management for the automatic tuning of an RC low-pass filter's cutoff frequency. The design uses 64-bit registers to represent real-valued parameters precisely, and integrates a uniform pseudo-random number generator for reproducible and

consistent hardware behavior. Additional enhancements such as linearly decreasing inertia weight, velocity constraints, and boundary-aware correction improve convergence stability and adaptability.

This architecture demonstrates a move toward intelligent, energy-efficient, self-tuning circuits capable of adjusting to varying operating conditions. By merging the adaptive capabilities of PSO with the precision and control of Verilog hardware, the proposed system lays the groundwork for next-generation mixed-signal designs that are fast, accurate, and autonomously optimized.

3.3 NOVELTY

The proposed system introduces a Dynamic Population Size Handling (DPSH) mechanism within a Verilog-based PSO framework—a capability not found in conventional HDL designs. This approach allows the hardware to intelligently activate or deactivate particle modules during execution, depending on optimization progress and system requirements. Such adaptive control effectively minimizes redundant computations, enhances hardware efficiency, and ensures better utilization of available resources. Consequently, the developed architecture delivers a faster, energy-efficient, and more reliable optimization engine specifically tailored for automatic tuning of electronic circuits.

3.4 IMPLEMENTATION FLOW

The implementation of the Dynamic Particle Selection Hybrid (DPSH)–based Particle Swarm Optimization (PSO) architecture follows a structured methodology covering design, integration, simulation, synthesis, and evaluation stages. Each step ensures functional correctness, modularity, and performance optimization of the system. The complete workflow is summarized below:

1. Design in Verilog

- The design consists of a single Verilog module responsible for the major functional units/tasks of the PSO algorithm — including Random Number Generation, Velocity Update, Position Update, and Fitness Computation. Each task operates on 64-bit real-equivalent values encoded through \$realtobits and \$bitstoreal conversions to enable arithmetic operations on analog-equivalent quantities.

- A Particle Bank structure is maintained to store individual particle positions (R and C values), velocities, and corresponding fitness scores. These are updated iteratively based on both local and global best solutions, ensuring proper information sharing across all particles.

- The Fitness Evaluation Unit computes the deviation between the target and computed cutoff frequencies, derived from

$$f_c = \frac{1}{2\pi RC}$$

This fitness value determines how close each particle is to the desired result.

- A PSO Controller task unit coordinates all particle updates per iteration. It triggers the random number generator, updates positions and velocities, and ensures proper iteration control.

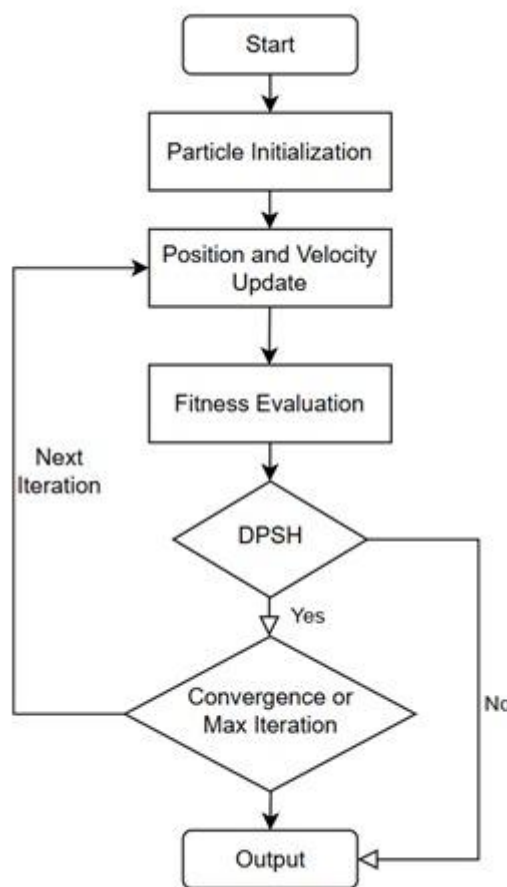


Fig. 3.4.1 Implementation Flow Chart

2. Simulation in Vivado

- Functional verification of the design is carried out using Xilinx Vivado through behavioral simulation.
- The simulations verify dynamic particle activation/deactivation behavior and the correct tracking of the global best (Gbest) parameter across iterations.
- The specific position and velocity (R, C, Fc, Error) of each particle and the particles disabled for the respective iteration are logged in the Console.

3.5 OBJECTIVES

- **Context**
 - Many modern engineering applications—such as analog filters, amplifiers, and digital systems—require efficient parameter optimization to enhance performance and reduce resource usage. The proposed Verilog-based **Particle Swarm Optimization (PSO)** with **Dynamic Population Size Handling (DPSH)** adaptively controls active particles, enabling faster convergence and improved hardware efficiency.
 - The design is applied to optimize parameters in analog filter tuning, amplifier biasing, and VLSI circuit design, and it can also be extended to signal processing and machine parameter estimation.
- **Expected outcomes:**

Simulations in **Xilinx Vivado** are expected to demonstrate faster convergence, reduced logic utilization, and accurate optimization results. The system aims to minimize percentage error in RC filter tuning while maintaining scalability and robustness.

CHAPTER-4

THEORY

4.1 Introduction

Analog circuit design often requires accurate control of frequency-dependent parameters such as cutoff frequency f_c bandwidth, and gain. In classical RC filter circuits, the cutoff frequency is defined by the relation

$$f_c = \frac{1}{2\pi RC}$$

where R and C represent the resistance and capacitance of the network respectively. The correct selection of these parameters determines the desired filtering characteristics. Traditionally, designers tune these parameters manually or through iterative simulation-based methods, which are both time-consuming and error-prone. Moreover, such manual tuning fails to adapt automatically to environmental variations, manufacturing tolerances, and long-term component drifts.

To address these challenges, this project employs Particle Swarm Optimization (PSO) — a population-based stochastic optimization technique inspired by social behaviour in bird flocking — to autonomously determine the optimal R and C values that achieve a user-defined cutoff frequency. The algorithm is implemented entirely in Verilog HDL, demonstrating how complex optimization processes can be mapped into hardware-level architectures suitable for FPGA or ASIC applications.

The PSO system evaluates multiple candidate solutions (R, C) pair. Through iterative position and velocity updates governed by individual and social knowledge, the particles collectively converge toward optimal combination minimizing the error:

$$E = \left| \frac{f_c - f_{\text{desired}}}{f_{\text{desired}}} \right| \times 100\%$$

By integrating PSO logic directly into Verilog, the system performs adaptive, on-chip tuning without requiring high-level software support, making it ideal for self-calibrating mixed-signal or adaptive analog systems.

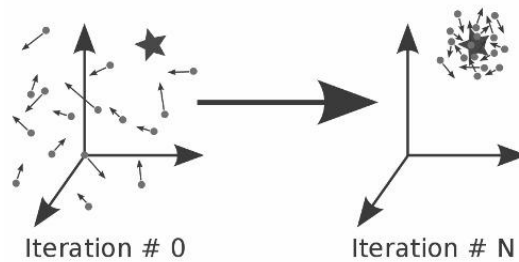


Fig. 4.1 Concept of PSO

4.2 Particle Swarm Optimization Fundamentals

PSO operates using a swarm of particles that explore the design space. Each particle maintains three key quantities:

- **Position** ($\vec{x}_i = [R_i, C_i]$) — the current candidate solution
- **Velocity** ($\vec{v}_i = [v_{R_i}, v_{C_i}]$) — the rate and direction of change
- **Personal best** ($\vec{p}_i$) and **global best** ($\vec{g}$) — best known local and global solutions

The algorithm iteratively updates velocity and position using the following equations:

$$\vec{v}_i(t+1) = w \cdot \vec{v}_i(t) + c_1 r_1 (\vec{p}_i - \vec{x}_i) + c_2 r_2 (\vec{g} - \vec{x}_i)$$

$$\vec{x}_i(t+1) = \vec{x}_i(t) + \vec{v}_i(t+1)$$

where,

w = inertia weight (linearly decreasing from 0.9 to 0.5),

c_1, c_2 = cognitive and social constants,

r_1, r_2 = random numbers between 0 and 1 ensuring stochastic exploration.

This dynamic adjustment allows the swarm to balance exploration (searching new regions) and exploitation (refining good solutions). The Verilog implementation adopts a floating-point encoding scheme using \$realtobits and \$bitstoreal for 64-bit precision arithmetic, preserving numerical accuracy across all iterations.

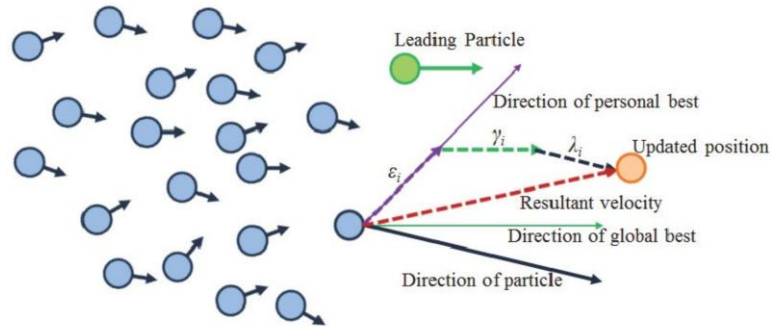


Fig. 4.2 Flow of Particles

4.3. Hardware Implementation Overview

The Fig 4.3 illustrates the overall PSO-based RC Auto-Tuning System Architecture. The module is functionally divided into the following tasks:

1. **Random Number Generator (RNG):** Produces uniform random numbers r_1 and r_2 required for stochastic velocity updates.
2. **Velocity Update Task:** Implements the PSO velocity equation using real-number arithmetic and dynamically adjusted inertia weight.
3. **Position Update Task:** Updates particle positions while enforcing reflecting boundary conditions to prevent invalid R, C values.
4. **Fitness Evaluation Unit:** Computes the resulting cutoff frequency f_c and percentage error relative to the target.
5. **Dynamic Particle Shutdown (DPSH) Controller:** Monitors the progress of each particle and deactivates those that fail to improve after several steps, thereby improving convergence efficiency.
6. **Global Best Selector:** Continuously compares particle fitness values and updates the best global solution.
7. **Repair Unit (Greedy Repair Logic):** Locally corrects particle positions by directly solving for R^* or C^* that exactly match the target frequency when feasible.

The Verilog module (pso_rc_tuner) integrates all these components, functions, tasks and coordinates the iterative optimization loop.

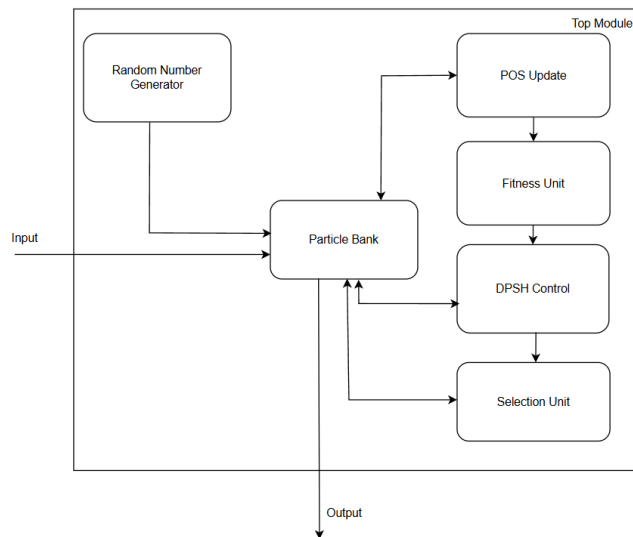


Fig. 4.3 Functional Diagram

4.4 Mathematical and Algorithmic Formulation

1. Cutoff Frequency Calculation

Each candidate pair (R, C) defines a cutoff frequency:

$$f_c = \frac{1}{2\pi RC}$$

The fitness function evaluates the deviation from the desired cutoff:

$$E = \left| \frac{f_c - f_{\text{desired}}}{f_{\text{desired}}} \right| \times 100$$

A smaller E value indicates a better candidate.

2. Edge Penalty Function

To prevent particles from stagnating at range limits, an **edge penalty** is introduced:

$$E_{\text{final}} = E + P(R, C)$$

where $P(R, C)$ increases proportionally when R or C approaches boundary margins $R_{\min}, R_{\max}, C_{\min}, C_{\max}$.

3. Dynamic Inertia Scheduling

The inertia term w decreases linearly with iteration count t :

$$w = w_{\text{start}} - (w_{\text{start}} - w_{\text{end}}) \frac{t}{N_{\text{iter}} - 1}$$

This mechanism gradually shifts the swarm from exploration to convergence.

Parameter	Description	Typical Value
N PARTICLES	Number of particles in the swarm	12
N ITER	Maximum number of iterations	25
F DESIRED	Target cutoff frequency	2.5 kHz
R MIN – R MAX	Resistance range	1 kΩ – 1 MΩ
C MIN – C MAX	Capacitance range	100 pF – 1 μF
C1, C2	Cognitive & social coefficients	1.6
W START – W END	Inertia range	0.9 – 0.5
EDGE MARGIN F	Edge margin factor	0.02
EDGE PENALTY PCT	Penalty factor	0.10
MAX WORSE STEPS	DPSH deactivation threshold	6

Table. 1 Parameter Specification for PSO Auto-Tuner

Signal Name	Direction	Width	Description
pos_R[i]	Internal Reg	64-bit	Current resistance value of particle i
pos_C[i]	Internal Reg	64-bit	Current capacitance value of particle i
vel_R[i]	Internal Reg	64-bit	Resistance velocity component
vel_C[i]	Internal Reg	64-bit	Capacitance velocity component
pbest_R[i], pbest_C[i]	Internal Reg	64-bit	Particle's best-known position
gbest_R_bits, gbest_C_bits	Output	64-bit	Global best R and C values
gbest_err_bits	Output	64-bit	Minimum achieved error
fc_b, err_b	Output	64-bit	Computed cutoff and corresponding error

Table 2. I/O Signal Description

Sample Simulation Output

Simulation in Xilinx Vivado confirmed rapid and stable convergence. Table illustrates error reduction versus iteration count. The swarm typically stabilizes within 10–15 iterations, achieving cutoff frequency accuracy better than 0.001% relative to the desired 2.5 kHz.

Iteration	Global Best Particle	(R) (Ω)	(C) (F)	(f _ c) (Hz)	Error (%)
0	P6	3280	19.5e-8	2473.6	1.06
5	P3	3120	20.2e-8	2496.2	0.15
10	P2	3182	19.9e-8	2499.3	0.03
20	P2	3183	19.9e-8	2500.1	0.004

Table 3. Sample Outputs

The DPSH mechanism ensures efficient convergence by suspending particles with poor performance, while the greedy repair unit fine-tunes near-optimal results. This hybrid of heuristic exploration and deterministic correction enhances both speed and precision.

The implemented PSO engine for RC cutoff frequency optimization merges evolutionary computation with hardware-level efficiency. By leveraging Verilog HDL for full algorithmic realization, the design demonstrates the potential of intelligent, adaptive analog parameter tuning directly in digital logic.

This approach provides a foundation for self-calibrating analog-digital hybrid systems, adaptive filters, and frequency-controlled embedded systems. The simulation results confirm that hardware-based PSO can achieve near-perfect frequency matching while maintaining computational simplicity and robustness.

CHAPTER 5

RESULTS AND CONCLUSION

5.1 RESULTS

```
Tcl Console

-----
Particle Swarm Optimization based RC Tuner using Dynamic Particle Size Handling
Inputs: F_DESIRED=46750.000000 Hz, R_MIN=100.000, R_MAX=100000000.000, C_MIN=1.000e-10, C_MAX=1.000e-06
Particles: 12, Iterations: 25
-----

P0: R=35823863.241 C=4.553e-07 fc=0.009759 err=99.999979%
P1: R=5171229.338 C=6.302e-07 fc=0.048835 err=99.999896%
P2: R=8538155.986 C=5.359e-07 fc=0.034781 err=99.999926%
P3: R=66608252.846 C=4.006e-07 fc=0.005964 err=99.999987%
P4: R=43645042.913 C=4.605e-07 fc=0.007919 err=99.999983%
P5: R=36096847.491 C=4.357e-07 fc=0.010121 err=99.999978%
P6: R=39601211.369 C=2.426e-07 fc=0.016564 err=99.999965%
P7: R=17814670.918 C=2.540e-07 fc=0.035170 err=99.999925%
P8: R=49649489.614 C=4.544e-07 fc=0.007054 err=99.999985%
P9: R=20829226.295 C=3.856e-07 fc=0.019816 err=99.999958%
P10: R=33988125.953 C=5.800e-07 fc=0.008074 err=99.999983%
P11: R=46546874.470 C=6.283e-07 fc=0.005442 err=99.999988%
-----

Iter 0: GBEST=P6 R=100.000 C=3.009e-07 fc=5289.604437 err=88.785338% | Disabled: none
Iter 1: GBEST=P1 R=62925.499 C=1.000e-10 fc=25292.599217 err=46.045042% | Disabled: none
Iter 2: GBEST=P1 R=62925.499 C=1.000e-10 fc=25292.599217 err=46.045042% | Disabled: none
Iter 3: GBEST=P1 R=981.433 C=3.469e-09 fc=46750.000000 err=0.141533% | Disabled: none
Iter 4: GBEST=P1 R=116.807 C=2.915e-08 fc=46750.000000 err=0.099999% | Disabled: none
Iter 5: GBEST=P1 R=116.807 C=2.915e-08 fc=46750.000000 err=0.099999% | Disabled: none
Iter 6: GBEST=P0 R=155.238 C=2.193e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 7: GBEST=P0 R=155.238 C=2.193e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 8: GBEST=P0 R=155.238 C=2.193e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 9: GBEST=P0 R=155.238 C=2.193e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 10: GBEST=P7 R=162.478 C=2.095e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 11: GBEST=P7 R=162.478 C=2.095e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 12: GBEST=P7 R=162.478 C=2.095e-08 fc=46750.000000 err=0.099997% | Disabled: P0, P3, P10
Iter 13: GBEST=P6 R=166.195 C=2.048e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 14: GBEST=P6 R=166.195 C=2.048e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 15: GBEST=P2 R=167.290 C=2.035e-08 fc=46750.000000 err=0.099997% | Disabled: P1
Iter 16: GBEST=P2 R=167.290 C=2.035e-08 fc=46750.000000 err=0.099997% | Disabled: P7
Iter 17: GBEST=P5 R=167.415 C=2.034e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 18: GBEST=P8 R=168.809 C=2.017e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 19: GBEST=P8 R=168.809 C=2.017e-08 fc=46750.000000 err=0.099997% | Disabled: P6
Iter 20: GBEST=P8 R=168.809 C=2.017e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 21: GBEST=P8 R=168.809 C=2.017e-08 fc=46750.000000 err=0.099997% | Disabled: P11
Iter 22: GBEST=P2 R=168.895 C=2.016e-08 fc=46750.000000 err=0.099997% | Disabled: none
Iter 23: GBEST=P2 R=168.895 C=2.016e-08 fc=46750.000000 err=0.099997% | Disabled: P5

Iter 24: GBEST=P2 R=168.895 C=2.016e-08 fc=46750.000000 err=0.099997% | Disabled: P8
-----
Final: GBEST=P2 R=168.895 C=2.016e-08 fc=46750.000000 Err=9.999656e-02%
```

Inputs: $f_{\text{desired}} = 46750\text{Hz}$, $100\Omega < R < 100\text{M}\Omega$, $100\text{pF} < C < 1\mu\text{F}$, Number of Particles = 12, Number of Iterations = 25

Outputs: $R=168.895\Omega$, $C = 20.16\text{nF}$, $f_c = 46750.00\text{ Hz}$ Error = 0.0999656%

5.2 CONCLUSION

The work successfully demonstrates the implementation of a Particle Swarm Optimization (PSO) engine in Verilog HDL for automatic tuning of an RC low-pass filter. By embedding optimization logic directly into hardware, the design eliminates the need for manual parameter adjustment and offers a reproducible, automated method for achieving a desired cutoff frequency. The use of 64-bit arithmetic ensured precision in handling real-valued computations, while the integrated pseudo-random number generator enabled randomized initialization and controlled stochastic behavior. Additional techniques such as velocity clamping, adaptive inertia, and dynamic particle shutdown contributed to faster convergence and better search stability.

Simulation results obtained through Xilinx Vivado confirmed the system's ability to find optimal or near-optimal resistance-capacitance pairs with minimal percentage error. The approach proved robust across a wide range of target frequencies, highlighting the practicality of PSO-based tuning in mixed-signal applications. Overall, the project establishes a compact and scalable framework for hardware-level optimization, paving the way for future advancements in self-calibrating analog-digital systems and intelligent electronic design automation.

5.3 FUTURE WORK

The existing PSO-based Tuned RC filter is functioning in the behavioral simulation environment, which is certainly a suitable environment for further physical realization; the next natural step is to push the design forward toward FPGA-based real-time optimization instead, allowing the PSO core to interface directly with an analog front-end using ADC and DAC modules connected to it. This would allow the application to watch the actual filter response in real-time and adjust component value dynamically, essentially creating true self-tuning analog systems. Fixed-point arithmetic implementations of the PSO engine could also be investigated to reduce hardware resources and increase speed, while still maintaining a reasonable level of accuracy.

Future developments may also consider multi-parameter, or multi-objective tuning where the PSO engine would tune not just cutoff frequency but other parameters like gain, phase margin, or bandwidth, all at the same time. Importantly, while this work addresses a standalone RC filter application, the concept could be extended to represent other analog circuits, whether they be active filters or amplifier networks, to demonstrate the scalability of this concept and implementation. Ultimately, this work offers the foundation for fully autonomous, hardware-embedded optimization frameworks that addresses the larger vision of smarter, self-correcting electronics, aimed at next generation mixed-signals and adaptive applications.

APPENDIX: (CODE)

Design code:

```
`timescale 1ns/1ps

// PSO-based auto-tuner for RC cutoff frequency (fc = 1/(2*pi*R*C)).
module pso_rc_tuner;

    // User parameters (unchanged functionality)
    parameter integer N_PARTICLES = 12;
    parameter integer N_ITER      = 25;
    parameter real   F_DESIREDD   = 2500.0; // Hz
    parameter real   R_MIN        = 1.0e3;  // Ohms
    parameter real   R_MAX        = 1.0e6;  // Ohms
    parameter real   C_MIN        = 100e-12; // Farads
    parameter real   C_MAX        = 1.0e-6;  // Farads
    parameter real   C1           = 1.6;    // cognitive
    parameter real   C2           = 1.6;    // social

    // Inertia schedule
    parameter real   W_START      = 0.9;
    parameter real   W_END        = 0.5;

    // Exploration/constraints (unchanged)
    parameter real   VCLAMP_R_F   = 0.10; // max |v_R| as fraction of R-range
    parameter real   VCLAMP_C_F   = 0.10; // max |v_C| as fraction of C-range
    parameter real   EDGE_MARGIN_F = 0.02; // 2% near edges
    parameter real   EDGE_PENALTY_PCT = 0.10; // up to +0.10% error when near edges
    parameter integer MAX_WORSE_STEPS = 6;
    parameter integer ENABLE_GREEDY_REPAIR = 1;
    localparam real TWO_PI = 6.283185307179586;

    // Particle state (64-bit "real" via bits)
    reg [63:0] pos_R   [0:N_PARTICLES-1];
    reg [63:0] pos_C   [0:N_PARTICLES-1];
    reg [63:0] vel_R   [0:N_PARTICLES-1];
    reg [63:0] vel_C   [0:N_PARTICLES-1];
    reg [63:0] pbest_R  [0:N_PARTICLES-1];
    reg [63:0] pbest_C  [0:N_PARTICLES-1];
    reg [63:0] pbest_err [0:N_PARTICLES-1];
    integer off_path_cnt [0:N_PARTICLES-1];
    integer active      [0:N_PARTICLES-1];

    // Global best
    reg [63:0] gbest_R_bits, gbest_C_bits, gbest_err_bits;
    integer gbest_idx;

    // Loop indices and temps
    integer i, j, it;
```

```

integer active_count, last_active_idx;
// Temps for math
reg [63:0] r1_b, r2_b;
reg [63:0] new_vr_b, new_vc_b;
reg [63:0] new_pr_b, new_pc_b;
reg [63:0] new_vr_adj_b, new_vc_adj_b;
reg [63:0] fc_b, err_b;
reg [63:0] fc_prev_b, err_prev_b;
real prev_best_err;
// Greedy repair temps
reg [63:0] fc_tmp_b, err_orig_b, err_rep_b;
reg [63:0] rep_R_b, rep_C_b;
// Iteration logging helpers
integer disabled_this_iter [0:N_PARTICLES-1];
integer cnt_before, act_before;
real gbest_err_before;
integer gbest_idx_before;
// Loop control temps
integer stop_now;
integer first;
// Final report
real Rf, Cf, fcf;
// Shared temp
reg [63:0] u_b;
real u, vr0, vc0;
// Gbest print temps
real gR, gC, gfc, gerr;
// Helpers: bit<->real
function real bits_to_real;
    input [63:0] b; begin bits_to_real = $bitstoreal(b); end
endfunction
function [63:0] real_to_bits;
    input real x; begin real_to_bits = $realtobits(x); end
endfunction
// RNG: uniform [0,1)
task rng01_bits;
    output [63:0] ubo;
    real ur;
    begin
        ur = $urandom / 4294967296.0; // [0,1)
        if (ur >= 1.0) ur = 0.999999999;
        ubo = real_to_bits(ur);
    end
endtask

```

```

    end
endtask

// RNG: uniform [lo,hi]
task rng_range_bits;
    input real lo, hi;
    output [63:0] out_b;
    reg [63:0] ub;
    real ur, v;
    begin
        rng01_bits(ub);
        ur = bits_to_real(ub);
        v = lo + ur * (hi - lo);
        out_b = real_to_bits(v);
    end
endtask

// Velocity update (dynamic inertia via it/N_ITER)
task velocity_update_bits;
    input [63:0] pos_R_b, pos_C_b, vel_R_b, vel_C_b, pbest_R_b, pbest_C_b, gbest_R_b, gbest_C_b, r1_bi,
r2_bi;
    output [63:0] vel_R_out_b, vel_C_out_b;
    real pr, pc, vr, vc, pbr, pbc, gbr, gbc, r1, r2, nvr, nvc;
    real iter_frac, W_CUR;
    begin
        pr = bits_to_real(pos_R_b);
        pc = bits_to_real(pos_C_b);
        vr = bits_to_real(vel_R_b);
        vc = bits_to_real(vel_C_b);
        pbr = bits_to_real(pbest_R_b);
        pbc = bits_to_real(pbest_C_b);
        gbr = bits_to_real(gbest_R_b);
        gbc = bits_to_real(gbest_C_b);
        r1 = bits_to_real(r1_bi);
        r2 = bits_to_real(r2_bi);
        if (N_ITER > 1) begin
            iter_frac = it / (N_ITER - 1.0);
            W_CUR = W_START - (W_START - W_END) * iter_frac;
        end else begin
            W_CUR = W_END;
        end
        nvr = W_CUR*vr + C1*r1*(pbr - pr) + C2*r2*(gbr - pr);
        nvc = W_CUR*vc + C1*r1*(pbc - pc) + C2*r2*(gbc - pc);
        vel_R_out_b = real_to_bits(nvr);

```



```

    vel_C_out_b = real_to_bits(nvc);
end
endtask
// Fitness: fc and percent error; includes small edge penalty
task fitness_bits;
    input [63:0] R_b, C_b;
    output [63:0] fc_bo, err_bo;
    real R, C, fc, de, perr;
    real r_norm, c_norm, pen;
    begin
        R = bits_to_real(R_b);
        C = bits_to_real(C_b);
        fc = 1.0 / (TWO_PI * R * C);
        de = (fc > F_DESIRED) ? (fc - F_DESIRED) : (F_DESIRED - fc);
        perr = (F_DESIRED > 0.0) ? (de / F_DESIRED * 100.0) : 1.0e300;
        // Edge penalty
        r_norm = (R - R_MIN) / (R_MAX - R_MIN);
        c_norm = (C - C_MIN) / (C_MAX - C_MIN);
        pen = 0.0;
        if (EDGE_MARGIN_F > 0.0) begin
            if (r_norm < EDGE_MARGIN_F)
                pen = pen + EDGE_PENALTY_PCT * (EDGE_MARGIN_F - r_norm) / EDGE_MARGIN_F;
            else if (r_norm > (1.0 - EDGE_MARGIN_F))
                pen = pen + EDGE_PENALTY_PCT * (r_norm - (1.0 - EDGE_MARGIN_F)) / EDGE_MARGIN_F;
            if (c_norm < EDGE_MARGIN_F)
                pen = pen + 0.5*EDGE_PENALTY_PCT * (EDGE_MARGIN_F - c_norm) / EDGE_MARGIN_F;
            else if (c_norm > (1.0 - EDGE_MARGIN_F))
                pen = pen + 0.5*EDGE_PENALTY_PCT * (c_norm - (1.0 - EDGE_MARGIN_F)) / EDGE_MARGIN_F;
        end
        perr = perr + pen;
        fc_bo = real_to_bits(fc);
        err_bo = real_to_bits(perr);
    end
endtask
// Global best from pbest across active particles
task update_gbest;
    integer jj;
    real best_e, e;
    integer found;
    begin
        best_e = 1.0e300;
        found = 0;

```

```

for (jj = 0; jj < N_PARTICLES; jj = jj + 1) begin
  if (active[jj]) begin
    e = bits_to_real(pbest_err[jj]);
    if (!found || (e < best_e)) begin
      best_e    = e;
      gbest_R_bits = pbest_R[jj];
      gbest_C_bits = pbest_C[jj];
      gbest_idx   = jj;
      found       = 1;
    end
  end
end

if (found) gbest_err_bits = real_to_bits(best_e);
end

endtask

// DPSH: deactivate after MAX_WORSE_STEPS consecutive worse steps
task dpsh_update;
  input [63:0] curr_err_b, pbest_err_prev_b;
  inout integer cnt, act;
  real ce, pe;
  begin
    ce = bits_to_real(curr_err_b);
    pe = bits_to_real(pbest_err_prev_b);
    if (ce > pe) cnt = cnt + 1; else cnt = 0;
    if (cnt >= MAX_WORSE_STEPS) act = 0;
  end
endtask

// Position update with reflecting bounds and velocity clamp
task position_update_bits;
  input [63:0] pos_R_b, pos_C_b, vel_R_b, vel_C_b;
  output [63:0] pos_R_out_b, pos_C_out_b, vel_R_out_b, vel_C_out_b;
  real pr, pc, vr, vc, nr, nc;
  real rspan, cspan, vmax_r, vmax_c;
  integer reflect_iter;
  begin
    pr = bits_to_real(pos_R_b);
    pc = bits_to_real(pos_C_b);
    vr = bits_to_real(vel_R_b);
    vc = bits_to_real(vel_C_b);
    rspan = (R_MAX - R_MIN);
    cspan = (C_MAX - C_MIN);
    vmax_r = VCLAMP_R_F * rspan;

```

```

vmax_c = VCLAMP_C_F * cspan;
if (vr > vmax_r) vr = vmax_r; else if (vr < -vmax_r) vr = -vmax_r;
if (vc > vmax_c) vc = vmax_c; else if (vc < -vmax_c) vc = -vmax_c;
nr = pr + vr;
nc = pc + vc;
for (reflect_iter = 0; reflect_iter < 4; reflect_iter = reflect_iter + 1) begin
  if (nr < R_MIN) begin
    nr = 2.0*R_MIN - nr; vr = -0.5*vr;
  end else if (nr > R_MAX) begin
    nr = 2.0*R_MAX - nr; vr = -0.5*vr;
  end
  if (nc < C_MIN) begin
    nc = 2.0*C_MIN - nc; vc = -0.5*vc;
  end else if (nc > C_MAX) begin
    nc = 2.0*C_MAX - nc; vc = -0.5*vc;
  end
  if ((nr >= R_MIN && nr <= R_MAX) && (nc >= C_MIN && nc <= C_MAX)) begin
    reflect_iter = 4; // exit
  end
end
if (nr < R_MIN) nr = R_MIN; if (nr > R_MAX) nr = R_MAX;
if (nc < C_MIN) nc = C_MIN; if (nc > C_MAX) nc = C_MAX;
pos_R_out_b = real_to_bits(nr);
pos_C_out_b = real_to_bits(nc);
vel_R_out_b = real_to_bits(vr);
vel_C_out_b = real_to_bits(vc);
end
endtask
// Greedy repair toward target fc for current R,C
task repair_choose_best;
input [63:0] Rin_b, Cin_b;
output [63:0] Rout_b, Cout_b;
real R_in, C_in, R1, C1loc, R2, C2loc, Cstar, Rstar;
reg [63:0] fc_b1, err_b1, fc_b2, err_b2;
real e1, e2;
begin
  R_in = bits_to_real(Rin_b);
  C_in = bits_to_real(Cin_b);
  // Candidate 1: fix R, solve C*
  Cstar = 1.0 / (TWO_PI * R_in * F_DESIRE);
  if (Cstar < C_MIN) C1loc = C_MIN; else if (Cstar > C_MAX) C1loc = C_MAX; else C1loc = Cstar;
  R1 = R_in;

```

```

// Candidate 2: fix C, solve R*
Rstar = 1.0 / (TWO_PI * C_in * F_DESIRE);
if (Rstar < R_MIN) R2 = R_MIN; else if (Rstar > R_MAX) R2 = R_MAX; else R2 = Rstar;
C2loc = C_in;
// Evaluate both
fitness_bits(real_to_bits(R1), real_to_bits(C1loc), fc_b1, err_b1);
fitness_bits(real_to_bits(R2), real_to_bits(C2loc), fc_b2, err_b2);
e1 = bits_to_real(err_b1);
e2 = bits_to_real(err_b2);
if (e1 <= e2) begin
    Rout_b = real_to_bits(R1);
    Cout_b = real_to_bits(C1loc);
end else begin
    Rout_b = real_to_bits(R2);
    Cout_b = real_to_bits(C2loc);
end
end
endtask
// Main
initial begin
    // Title and inputs
    $display("-----");
    $display("Particle Swarm Optimization based RC Tuner using Dynamic Particle Size Handling");
    $display("Inputs: F_DESIRE=%.6f Hz, R_MIN=%.3f, R_MAX=%.3f, C_MIN=%.3e, C_MAX=%.3e",
F_DESIRE, R_MIN, R_MAX, C_MIN, C_MAX);
    $display("Particles: %0d, Iterations: %0d", N_PARTICLES, N_ITER);
    $display("-----");
    // Initialize particles and print initial states
    for (i = 0; i < N_PARTICLES; i = i + 1) begin
        rng_range_bits(R_MIN, R_MAX, pos_R[i]);
        rng_range_bits(C_MIN, C_MAX, pos_C[i]);
        // small random initial velocity within ±5% of range
        rng01_bits(u_b); u = 2.0*bits_to_real(u_b) - 1.0; vr0 = 0.05*(R_MAX-R_MIN)*u;
        rng01_bits(u_b); u = 2.0*bits_to_real(u_b) - 1.0; vc0 = 0.05*(C_MAX-C_MIN)*u;
        vel_R[i] = real_to_bits(vr0);
        vel_C[i] = real_to_bits(vc0);
        pbest_R[i] = pos_R[i];
        pbest_C[i] = pos_C[i];
        fitness_bits(pos_R[i], pos_C[i], fc_b, err_b);
        pbest_err[i] = err_b;
        off_path_cnt[i] = 0;

```

```

    active[i] = 1;
    // Initial state line
    $display("P%0d: R=%.3f C=%.3e fc=%.6f err=%.6f%%",
        i, bits_to_real(pos_R[i]), bits_to_real(pos_C[i]),
        bits_to_real(fc_b), bits_to_real(err_b));
end
// Initial global best
update_gbest();
$display("----");
// Iterations
for (it = 0; it < N_ITER; it = it + 1) begin
    // clear disabled list for this iteration
    for (j = 0; j < N_PARTICLES; j = j + 1) disabled_this_iter[j] = 0;
    active_count = 0;
    last_active_idx = -1;
    // update all particles
    for (i = 0; i < N_PARTICLES; i = i + 1) begin
        if (active[i]) begin
            // Random coefficients
            rng01_bits(r1_b);
            rng01_bits(r2_b);
            // Velocity and position
            velocity_update_bits(pos_R[i], pos_C[i], vel_R[i], vel_C[i], pbest_R[i], pbest_C[i], gbest_R_bits,
gbest_C_bits, r1_b, r2_b, new_vr_b, new_vc_b);
            position_update_bits(pos_R[i], pos_C[i], new_vr_b, new_vc_b, new_pr_b, new_pc_b, new_vr_adj_b,
new_vc_adj_b);
            // Greedy repair (optional)
            if (ENABLE_GREEDY_REPAIR) begin
                fitness_bits(new_pr_b, new_pc_b, fc_tmp_b, err_orig_b);
                repair_choose_best(new_pr_b, new_pc_b, rep_R_b, rep_C_b);
                fitness_bits(rep_R_b, rep_C_b, fc_tmp_b, err_rep_b);
                if (bits_to_real(err_rep_b) < bits_to_real(err_orig_b)) begin
                    new_pr_b = rep_R_b;
                    new_pc_b = rep_C_b;
                end
            end
            vel_R[i] = new_vr_adj_b; vel_C[i] = new_vc_adj_b;
            pos_R[i] = new_pr_b; pos_C[i] = new_pc_b;
            // Fitness and DPSH/pbest
            fitness_bits(pos_R[i], pos_C[i], fc_b, err_b);
            // DPSH around update
            cnt_before = off_path_cnt[i];

```

```

    act_before = active[i];
    prev_best_err = bits_to_real(pbest_err[i]);
    dpsh_update(err_b, pbest_err[i], off_path_cnt[i], active[i]);
    if (active[i] && (bits_to_real(err_b) < prev_best_err)) begin
        pbest_err[i] = err_b;
        pbest_R[i] = pos_R[i];
        pbest_C[i] = pos_C[i];
    end
    if (act_before && !active[i]) begin
        disabled_this_iter[i] = 1;
    end
    if (active[i]) begin
        active_count = active_count + 1;
        last_active_idx = i;
    end
end
end
// Decide GBEST for this iteration and whether to stop
stop_now = 0;
if (active_count == 1) begin
    fitness_bits(pos_R[last_active_idx], pos_C[last_active_idx], fc_b, err_b);
    gbest_R_bits = pos_R[last_active_idx];
    gbest_C_bits = pos_C[last_active_idx];
    gbest_err_bits = err_b;
    gbest_idx = last_active_idx;
    stop_now = 1;
end else begin
    update_gbest();
    if (active_count == 0) stop_now = 1;
end
// Print iteration summary: GBEST and disabled list
gR = bits_to_real(gbest_R_bits);
gC = bits_to_real(gbest_C_bits);
gfc = 1.0 / (TWO_PI * gR * gC);
gerr = bits_to_real(gbest_err_bits);
$write("Iter %0d: GBEST=P%0d R=%.3f C=%.3e fc=%.6f err=%.6f%% | Disabled:", it, gbest_idx, gR, gC,
gfc, gerr);
first = 1;
for (j = 0; j < N_PARTICLES; j = j + 1) begin
    if (disabled_this_iter[j]) begin
        if (!first) $write(",");
        $write(" P%0d", j);
    end
end

```

```

        first = 0;
    end
end
if (first) $write(" none");
$display("");
if (stop_now) begin
    it = N_ITER; // exit after this iteration
end
end
// Final report
$display("-----");
Rf = bits_to_real(gbest_R_bits);
Cf = bits_to_real(gbest_C_bits);
fcf = 1.0 / (TWO_PI * Rf * Cf);
$display("Final: GBEST=P%0d R=%.3f C=%.3e fc=%.6f Err=%.6e%%", gbest_idx, Rf, Cf, fcf,
bits_to_real(gbest_err_bits));
$display("-----");
$finish;
end
endmodule

```

TestBench:

```

`timescale 1ns/1ps
module tb_pso_rc_tuner;
    pso_rc_tuner #(
        .N_PARTICLES(12),
        .N_ITER(25),
        .F_DESIRED(46750.0),
        .R_MIN(1.0e2),
        .R_MAX(1.0e8),
        .C_MIN(100e-12),
        .C_MAX(1.0e-6),
        .C1(1.6),
        .C2(1.6),
        .W_START(0.7),
        .W_END(0.7)
    ) dut ();
    initial begin
        #1000000000;
        $display("Timeout: stopping simulation");
        $finish;
    end
endmodule

```

REFERENCES

- [1] M. Ravindran, S. Sarbaras, S. Selvam, and M. Sridharan, "FPGA Implementation and Validation of Particle Swarm Optimization suited for varied applications," *2023 International Conference on Computer, Electronics & Electrical Engineering & their Applications (IC2E3), Srinagar Garhwal, India, 2023*, pp. 1–6, doi: 10.1109/IC2E357697.2023.10262426.
- [2] J. Kennedy and R. Eberhart. "Particle swarm optimization". In: *Proceedings of ICNN'95-International Conference on Neural Networks*. Vol. 4. 1995, 1942– 1948 vol.4. DOI: 10.1109/ICNN.1995.488968.
- [3] M.-P. Song and G.-C. Gu, "Research on particle swarm optimization: a review," *Proceedings of 2004 International Conference on Machine Learning and Cybernetics (IEEE Cat. No.04EX826)*, vol. 4, pp. 2236–2241, 2004, doi: 10.1109/ICMLC.2004.1382171.
- [4] P. -Y. Kuo and L. -F. Sie, "Analyze the behavior model based on Verilog-A for Sallen-Key low-pass filter," *2015 IEEE International Conference on Consumer Electronics - Taiwan, Taipei, Taiwan, 2015*, pp. 460-461, doi: 10.1109/ICCE-TW.2015.7216998.
- [5] F. Shaikh, I. H. Kalwar, T. D. Memon and S. Sheikh, "Design and analysis of linear phase FIR filter in FPGA using PSO algorithm," *2017 6th Mediterranean Conference on Embedded Computing (MECO)*, Bar, Montenegro, 2017, pp. 1-4, doi: 10.1109/MECO.2017.7977216.